

Lista de tareas — Machine Learning

Soluciones

Esteban García

19 de julio de 2026

1. Hoeffding no aplica: el experimento de las monedas (Ej. 1.10)

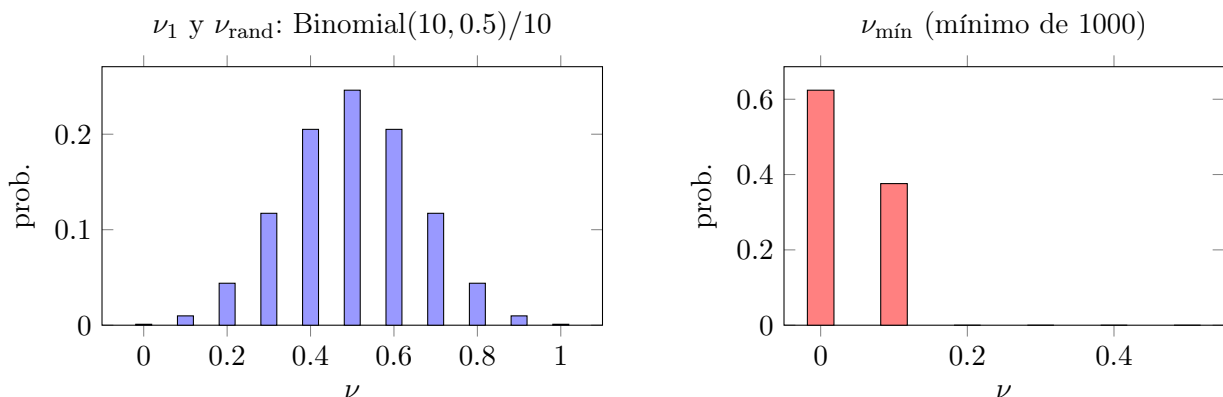
Lanzamos 1000 monedas justas, cada una 10 veces. Miramos tres de ellas: c_1 (la primera), c_{rand} (una al azar) y $c_{\text{mín}}$ (la que sacó *menos* caras). Sus fracciones de caras son ν_1 , ν_{rand} , $\nu_{\text{mín}}$.

Solución. (a) ¿Cuánto vale μ para cada moneda? Todas son monedas justas, así que la probabilidad real de cara es $\mu = 0.5$ para las tres. Ojo con esto: μ es una propiedad de la moneda, no de *cómo* la elegimos. Aunque $c_{\text{mín}}$ la escojamos mirando los datos, la moneda en sí sigue siendo justa: $\mu = 0.5$.

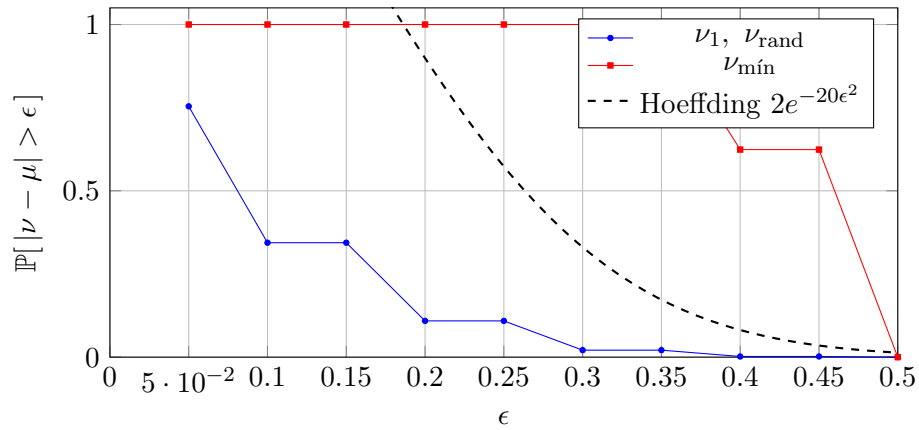
(b) Histogramas. Como cada ν es (número de caras)/10 con 10 lanzamientos justos, ν_1 y ν_{rand} siguen una Binomial(10, 0.5)/10, centrada en 0.5:

$$\mathbb{P}\left[\nu = \frac{k}{10}\right] = \binom{10}{k} \frac{1}{2^{10}}, \quad k = 0, \dots, 10.$$

En cambio $\nu_{\text{mín}}$ es el *mínimo* de 1000 de esas variables, así que se apelmaza cerca de 0. Un cálculo rápido: $\mathbb{P}[\nu_{\text{mín}} = 0] = 1 - (1 - 2^{-10})^{1000} \approx 0.62$ y casi todo el resto cae en $\nu_{\text{mín}} = 0.1$. Es decir, $\nu_{\text{mín}} \in \{0, 0.1\}$ casi siempre.



(c) $\mathbb{P}[|\nu - \mu| > \epsilon]$ frente a la cota de Hoeffding $2e^{-2\epsilon^2 N}$ (con $N = 10$). Comparando la frecuencia empírica de cada moneda con la cota:



(d) ¿Quién cumple la cota y quién no? c_1 y c_{rand} la cumplen holgadamente (sus curvas van por debajo de la línea punteada). $c_{\text{mín}}$ *no* la cumple: su curva se queda pegada en 1 mucho más allá de lo que Hoeffding permite. La razón es la clave de todo el tema: la cota de Hoeffding vale para una hipótesis *fijada de antemano*. c_1 está fija (es la primera) y c_{rand} se elige sin mirar los resultados, así que ambas son “bins” honestos. Pero $c_{\text{mín}}$ se elige *después* de ver los datos, escogiendo justo la de peor comportamiento: eso es hacer trampa estadística, y la cota de un solo bin deja de aplicar.

(e) **Relación con los múltiples bins.** Elegir $c_{\text{mín}}$ es exactamente lo que hace el aprendizaje: seleccionar la hipótesis g que mejor se ajusta a la muestra de entre M candidatas. Para esa selección hay que pagar el *union bound* sobre todos los bins:

$$\mathbb{P}[|\nu_{\text{mín}} - \mu| > \epsilon] \leq M \cdot 2e^{-2\epsilon^2 N}, \quad M = 1000.$$

Por eso, cuando M crece (o es infinito), Hoeffding solo ya no basta y aparece la función de crecimiento $m_{\mathcal{H}}(N)$ y la dimensión VC. El código que genera todo lo anterior:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def one_run(n_coins=1000, n_flips=10, rng=None):
5     rng = rng or np.random.default_rng()
6     heads = rng.binomial(n_flips, 0.5, size=n_coins) # caras por moneda
7     nu = heads / n_flips
8     return nu[0], nu[rng.integers(n_coins)], nu[np.argmin(heads)]
9
10 N = 100_000
11 res = np.array([one_run() for _ in range(N)])
12 nu1, nurand, numin = res[:,0], res[:,1], res[:,2]
13
14 # (b) histogramas
15 for data, name in [(nu1, "nu_1"), (nurand, "nu_rand"), (numin, "nu_min")]:
16     plt.figure(); plt.hist(data, bins=np.arange(-0.05, 1.15, 0.1), density=True)
17     plt.title(name); plt.xlabel("nu")
18
19 # (c) P[|nu-mu|>eps] vs eps, con mu = 0.5
20 eps = np.linspace(0, 0.5, 51)
21 tail = lambda d: np.array([np.mean(np.abs(d-0.5) > e) for e in eps])
22 plt.figure()
23 for d,l in [(nu1, "nu_1"), (nurand, "nu_rand"), (numin, "nu_min")]:
24     plt.plot(eps, tail(d), label=l)
25 plt.plot(eps, 2*np.exp(-2*10*eps**2), "k--", label="Hoeffding")
26 plt.legend(); plt.xlabel("eps"); plt.ylabel("P[|nu-mu|>eps]")
27 plt.show()

```

2. Entropía: por qué $H(X) = -\sum_i P(X=i) \log_2 P(X=i)$

Solución. Paso 1: la información de un evento de probabilidad p es $-\log_2 p$ bits. Pensémoslo como el juego de las “20 preguntas”. Cada pregunta de sí/no parte el espacio de posibilidades en dos. Si un mensaje tiene probabilidad $p = 1/2^k$, entonces está “escondido” entre 2^k opciones igualmente probables, y para aislarlo necesito $k = \log_2(1/2^k) = -\log_2 p$ preguntas (cada pregunta bien hecha reduce el espacio a la mitad). Definimos entonces el *contenido de información*

$$I(p) = -\log_2 p \quad [\text{bits}].$$

Esta forma no es arbitraria: si pedimos que la información sea aditiva para eventos independientes, $I(p_1 p_2) = I(p_1) + I(p_2)$, la única función continua que lo cumple es $I(p) = -c \log p$, y fijando la base 2 tenemos $c = 1$ y unidades en bits.

Paso 2: formalizándolo con Kraft (codificación óptima). Queremos un código binario libre de prefijos que asigne a cada símbolo i una palabra de longitud ℓ_i . La desigualdad de Kraft dice que esto es posible si y solo si $\sum_i 2^{-\ell_i} \leq 1$. Buscamos las longitudes que minimizan la longitud media $L = \sum_i p_i \ell_i$:

$$\min_{\ell_i} \sum_i p_i \ell_i \quad \text{s.a.} \quad \sum_i 2^{-\ell_i} \leq 1.$$

Con un multiplicador de Lagrange (y relajando ℓ_i a reales) se obtiene $\ell_i^* = -\log_2 p_i$, que justo satura Kraft ($\sum_i 2^{\log_2 p_i} = \sum_i p_i = 1$). Sustituyendo:

$$L^* = \sum_i p_i \ell_i^* = -\sum_i p_i \log_2 p_i = H(X).$$

Paso 3: la entropía es la información esperada. La entropía no es más que el promedio del contenido de información sobre la distribución:

$$H(X) = \mathbb{E}[I(P(X))] = \mathbb{E}[-\log_2 P(X)] = -\sum_i P(X=i) \log_2 P(X=i).$$

En palabras: $H(X)$ es el número *promedio* de preguntas binarias (o bits) que se necesitan para identificar el resultado de X , y ese promedio se minimiza justo cuando cada símbolo usa $-\log_2 p_i$ bits. Por eso la fórmula tiene esa forma.

(Extra) El mismo resultado por conteo (la derivación de Boltzmann/Stirling). Esta es, probablemente, la versión que espera tu curso. Toma un mensaje larguísimo de N símbolos; el símbolo i aparece $N_i \approx N p_i$ veces. El número de mensajes distintos con esas frecuencias es el coeficiente multinomial

$$W = \frac{N!}{N_1! N_2! \cdots N_n!}.$$

Para señalar *cuál* de esos W mensajes tienes necesitas $\log_2 W$ bits, así que la información *por símbolo* es $\frac{1}{N} \log_2 W$. Usando Stirling ($\ln m! \approx m \ln m - m$):

$$\begin{aligned} \frac{1}{N} \log_2 W &= \frac{1}{N} \left(\log_2 N! - \sum_i \log_2 N_i! \right) \\ &\approx \frac{1}{N} \left(N \log_2 N - N - \sum_i (N_i \log_2 N_i - N_i) \right) \\ &= \frac{1}{N} \left(N \log_2 N - \sum_i N p_i \log_2 (N p_i) \right) \\ &= \log_2 N - \sum_i p_i (\log_2 N + \log_2 p_i) = -\sum_i p_i \log_2 p_i = H(X), \end{aligned}$$

donde usamos $\sum_i N_i = N$ y $\sum_i p_i = 1$. Es decir: la entropía es, literalmente, el logaritmo del número de mensajes típicos, por símbolo.

3. Support Vector Machine y optimización convexa

Solución. Tenemos datos separables $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ con $y_i \in \{-1, +1\}$ y queremos el hiperplano $\mathbf{w}^\top \mathbf{x} + b = 0$ que los separa *con el mayor margen posible*.

Del margen a la optimización. La distancia de un punto al hiperplano es $|\mathbf{w}^\top \mathbf{x}_i + b| / \|\mathbf{w}\|$. Como podemos reescalar $(\mathbf{w}, b)$ libremente, fijamos la escala pidiendo que el punto más cercano cumpla $|\mathbf{w}^\top \mathbf{x}_i + b| = 1$ (forma canónica). Entonces el margen vale $1 / \|\mathbf{w}\|$ y maximizarlo equivale a minimizar $\|\mathbf{w}\|^2$. El problema *primal* (margen duro) es

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.a.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, N.$$

Esto es un **programa cuadrático convexo**: objetivo convexo (cuadrático definido positivo) y restricciones afines. Por lo tanto tiene mínimo global único y no hay mínimos locales que nos engañen.

Lagrangiano y dual. Con multiplicadores $\alpha_i \geq 0$,

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1].$$

Las condiciones de estacionariedad (KKT) dan

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i, \quad \frac{\partial \mathcal{L}}{\partial b} = 0 \Rightarrow \sum_i \alpha_i y_i = 0.$$

Sustituyendo se obtiene el problema *dual*, que solo depende de productos internos:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.a.} \quad \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0.$$

También es convexo. Como se cumple la condición de Slater, hay *dualidad fuerte*: resolver el dual resuelve el primal.

Vectores de soporte. Por complementariedad ($\alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1] = 0$), solo los puntos que están *justo sobre el margen* tienen $\alpha_i > 0$. Esos son los vectores de soporte: son los únicos que definen $\mathbf{w}$.

Margen blando y kernels. Si los datos no son perfectamente separables, se añaden holguras $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.a.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0,$$

cuyo dual es idéntico salvo que ahora $0 \leq \alpha_i \leq C$. Y como todo depende de $\mathbf{x}_i^\top \mathbf{x}_j$, podemos cambiar ese producto por un kernel $K(\mathbf{x}_i, \mathbf{x}_j)$ para separar en espacios no lineales, sin perder la convexidad.

Código: la implementación desde cero (descenso de subgradiente) está en el archivo `SVM_desde_cero.py`, dentro de la carpeta `Notebooks`.

4. Perceptrón: implementación, convergencia y Ej. 1.2–1.3

Solución. Implementación (PLA). La regla de actualización es la de la ecuación (1.3): si $\mathbf{x}(t)$ está mal clasificado, $\mathbf{w}(t+1) = \mathbf{w}(t) + y(t) \mathbf{x}(t)$.

```
1 import numpy as np
2
3 def pla(X, y, max_iter=100000):
4     Xb = np.hstack([np.ones((len(X),1)), X]) # x0 = 1 (sesgo)
5     w = np.zeros(Xb.shape[1])
6     for t in range(max_iter):
```

```

7     pred = np.sign(Xb @ w); pred[pred == 0] = -1
8     mis = np.where(pred != y)[0]
9     if len(mis) == 0:
10         return w, t                                # separ los datos
11     i = mis[0]                                       # (o np.random.choice(mis))
12     w = w + y[i] * Xb[i]                           # regla (1.3)
13     return w, max_iter

```

Ejercicio 1.2 (perceptrón antispam).

- Palabras con *peso positivo grande*: las muy asociadas a spam, como “gratis”, “premio”, “viagra”, “\$\$\$”, “oferta”, “clic aquí”.
- Palabras con *peso negativo*: las típicas de correo legítimo para ese usuario: su nombre, “reunión”, “factura”, nombres de proyectos o colegas.
- El parámetro que controla cuántos mensajes *fronterizos* terminan marcados como spam es el *umbral* (el sesgo $b = w_0$). Bajarlo hace al filtro más agresivo (más spam); subirlo, más permisivo.

Ejercicio 1.3 (la actualización va “en la dirección correcta”). Sea $\mathbf{x}(t)$ mal clasificado y $\mathbf{w}(t+1) = \mathbf{w}(t) + y(t)\mathbf{x}(t)$.

- Como está mal clasificado, $\text{sign}(\mathbf{w}^\top(t)\mathbf{x}(t)) \neq y(t)$, luego $y(t)\mathbf{w}^\top(t)\mathbf{x}(t) < 0$.
- Calculamos:

$$y(t)\mathbf{w}^\top(t+1)\mathbf{x}(t) = y(t)(\mathbf{w}(t) + y(t)\mathbf{x}(t))^\top \mathbf{x}(t) = y(t)\mathbf{w}^\top(t)\mathbf{x}(t) + \underbrace{y(t)^2 \|\mathbf{x}(t)\|^2}_{=1}.$$

Como $\|\mathbf{x}(t)\|^2 > 0$, queda $y(t)\mathbf{w}^\top(t+1)\mathbf{x}(t) > y(t)\mathbf{w}^\top(t)\mathbf{x}(t)$.

- La cantidad $y(t)\mathbf{w}^\top \mathbf{x}(t)$ (la que debe volverse positiva para clasificar bien) *aumenta* tras la actualización. No garantiza acertar en un solo paso, pero empuja al punto hacia el lado correcto: por eso es “un movimiento en la dirección correcta”.

Preguntas adicionales.

- ¿Encuentra los parámetros correctos (separa los datos)? Sí, **siempre que los datos sean linealmente separables**. Si no lo son, el PLA nunca converge (oscila).
- ¿Para en un número finito de pasos? Sí, y lo garantiza el siguiente teorema.
- ¿Por qué funciona? Es la consecuencia del Ej. 1.3: cada corrección alinea $\mathbf{w}$ con la solución. La demostración lo formaliza.

Teorema 1 (Convergencia del perceptrón, Novikoff). *Si existe $\mathbf{w}^*$ con $\|\mathbf{w}^*\| = 1$ y margen $\gamma = \min_i y_i(\mathbf{w}^{*\top} \mathbf{x}_i) > 0$, y $R = \max_i \|\mathbf{x}_i\|$, entonces el PLA comete a lo sumo R^2/γ^2 actualizaciones.*

Demostración. Sea $\mathbf{w}(t)$ tras t actualizaciones (arrancando en $\mathbf{w}(0) = \mathbf{0}$).

Cota inferior (el numerador crece lineal). En cada corrección con un punto mal clasificado,

$$\mathbf{w}^{*\top} \mathbf{w}(t+1) = \mathbf{w}^{*\top} \mathbf{w}(t) + y_i \mathbf{w}^{*\top} \mathbf{x}_i \geq \mathbf{w}^{*\top} \mathbf{w}(t) + \gamma,$$

así que $\mathbf{w}^{*\top} \mathbf{w}(t) \geq t\gamma$.

Cota superior (el denominador crece a lo sumo lineal). Como el punto estaba mal clasificado, $y_i \mathbf{w}^\top(t) \mathbf{x}_i \leq 0$, luego

$$\|\mathbf{w}(t+1)\|^2 = \|\mathbf{w}(t)\|^2 + 2y_i \mathbf{w}^\top(t) \mathbf{x}_i + \|\mathbf{x}_i\|^2 \leq \|\mathbf{w}(t)\|^2 + R^2,$$

de donde $\|\mathbf{w}(t)\|^2 \leq tR^2$.

Juntando. Por Cauchy–Schwarz, $t\gamma \leq \mathbf{w}^{\star\top} \mathbf{w}(t) \leq \|\mathbf{w}(t)\| \leq \sqrt{t}R$, es decir $t\gamma \leq \sqrt{t}R$, y por tanto

$$t \leq \frac{R^2}{\gamma^2}.$$

El número de actualizaciones es finito (no depende de N ni de la dimensión), lo que responde (b) y, junto con el Ej. 1.3, el (c). $\square$

¿Y si los datos NO son separables? Entonces el PLA no converge: se la pasa oscilando (nunca llega a cero errores que lo detengan). El arreglo práctico es el **algoritmo Pocket**: corres el PLA de siempre, pero vas guardando “en el bolsillo” los mejores pesos vistos hasta el momento (los de menor error sobre la muestra) y al final devuelves esos. No garantiza el óptimo, pero te entrega la mejor solución encontrada y es robusto al ruido.

```

1 def pocket(X, y, epochs=1000):
2     Xb = np.hstack([np.ones((len(X),1)), X])
3     w = np.zeros(Xb.shape[1]); best_w = w.copy()
4     best_err = np.mean(np.sign(Xb @ w) != y)
5     for _ in range(epochs):
6         pred = np.sign(Xb @ w); pred[pred == 0] = -1
7         mis = np.where(pred != y)[0]
8         if len(mis) == 0:
9             return w # separable: es el PLA clasico
10        i = np.random.choice(mis)
11        w = w + y[i] * Xb[i] # paso del PLA
12        err = np.mean(np.sign(Xb @ w) != y)
13        if err < best_err: # mejoro? guardalo en el bolsillo
14            best_err, best_w = err, w.copy()
15    return best_w

```

Código: la implementación del PLA sobre dos distribuciones gaussianas separables está en el archivo `Perceptron_gaussianas.py`, dentro de la carpeta `Notebooks`.

5. XOR con neuronas básicas y con perceptrones

Solución. Un solo perceptrón no puede con el XOR (no es linealmente separable). La salida se logra combinando compuertas. Usamos la neurona de umbral (McCulloch–Pitts) con entradas en $\{0, 1\}$ y salida 1 si $\mathbf{w}^\top \mathbf{x} + b \geq 0$, y 0 si no.

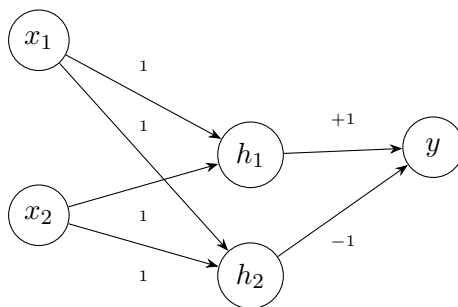
Idea lógica. XOR = OR – AND: es 1 cuando al menos una entrada es 1 (OR) pero *no* cuando ambas lo son (AND). Con dos neuronas ocultas y una de salida:

$$h_1 = \text{OR}(x_1, x_2) = \mathbf{1}[x_1 + x_2 - 0.5 \geq 0], \quad h_2 = \text{AND}(x_1, x_2) = \mathbf{1}[x_1 + x_2 - 1.5 \geq 0],$$

$$y = \mathbf{1}[h_1 - h_2 - 0.5 \geq 0].$$

Es decir, pesos ($\mathbf{w}_{h_1} = (1, 1)$, $b_{h_1} = -0.5$), ($\mathbf{w}_{h_2} = (1, 1)$, $b_{h_2} = -1.5$) y ($\mathbf{w}_y = (1, -1)$, $b_y = -0.5$). Comprobación:

x_1	x_2	h_1 (OR)	h_2 (AND)	y (XOR)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0



Así, “conectando varias neuronas básicas” (dos en la capa oculta con umbrales distintos y una de salida) el XOR queda modelado con pesos concretos para cada perceptrón. La moraleja: lo que un perceptrón solo no puede, dos capas sí.

6. Reto Netflix vía factorización no negativa (KDD Cup 2007)

Solución. El reto. La Tarea 1 del KDD Cup 2007, “*Who Rated What*”, usaba los mismos datos del Netflix Prize. No pedía *qué* calificación daría un usuario, sino la *probabilidad de que un usuario calificara una película* durante 2006, para una lista de 100,000 pares (usuario, película). El reporte ganador (Kurucz, Benczúr, Kiss y colegas, SZTAKI) combinó SVD, correlaciones y minería de secuencias frecuentes, pero el motor central fue una **factorización de matrices no negativa (NMF)** de bajo rango y ponderada, tratando las entradas no observadas como variables a optimizar.

Qué hace la NMF. Tenemos la matriz $R \in \mathbb{R}^{U \times M}$ (usuarios $\times$ películas). La NMF la aproxima como producto de dos matrices *no negativas* de rango bajo K :

$$R \approx WH, \quad W \in \mathbb{R}_{\geq 0}^{U \times K}, \quad H \in \mathbb{R}_{\geq 0}^{K \times M}, \quad \min_{W, H \geq 0} \|R - WH\|_F^2.$$

El paralelo Temas–Temáticas–Películas (la sugerencia). La dimensión latente K son las “temáticas” (piénsalas como géneros o estilos ocultos: acción noventera, drama romántico, etc.). Entonces:

- W conecta **usuarios** $\rightarrow$ **temáticas**: W_{uk} = cuánto le atrae al usuario u la temática k .
- H conecta **temáticas** $\rightarrow$ **películas**: H_{km} = cuánto pertenece la película m a la temática k .
- La composición reconstruye **usuarios** $\rightarrow$ **películas**: $\hat{R}_{um} = \sum_k W_{uk} H_{km}$, la afinidad predicha del usuario por la película.

La restricción de *no negatividad* es la que hace que los factores sean interpretables como una *suma de partes*: una película es una mezcla no negativa de temáticas y un usuario es una mezcla no negativa de gustos. No hay “cancelaciones” raras. Así, aunque solo observamos unas pocas entradas de R , la NMF descubre las temáticas que mejor explican el patrón de quién ve qué, y con eso se puede predecir la propensión de un usuario a calificar (ver) una película que no había tocado: si el usuario carga fuerte en las mismas temáticas que la película, la probabilidad sube. Esa es la esencia de cómo la factorización “resuelve” el reto Netflix.

Fuentes: https://kdd.org/exploration_files/1-task-1-winner.pdf | <https://kdd.org/kdd-cup/view/kdd-cup-2007/Tasks>

7. K Nearest Neighbours: conjunto hipótesis y algoritmo

Solución. Conjunto hipótesis. KNN es *no paramétrico* y *basado en instancias*. No hay un vector de pesos que ajustar: dada la muestra D y el k , la hipótesis para una consulta $\mathbf{x}$ es el voto

mayoritario (clasificación) o el promedio (regresión) de sus k vecinos más cercanos. El conjunto hipótesis es, por tanto, el conjunto de todas las funciones *constantes a trozos* realizables de esa forma:

$$\mathcal{H} = \{g_{D,k} : \mathbf{x} \mapsto \text{voto de los } k \text{ vecinos de } \mathbf{x} \text{ en } D\}.$$

Para $k = 1$, las fronteras de decisión son las del *diagrama de Voronoi* de los puntos de entrenamiento, y la hipótesis puede ajustar la muestra a la perfección ($E_{in} = 0$). Es un $\mathcal{H}$ enorme y flexible: su capacidad la controla k (con k pequeño, mucha varianza y poco sesgo; con k grande, más suave y con más sesgo).

Algoritmo de aprendizaje. Es un aprendiz *perezoso* (*lazy*): “entrenar” consiste simplemente en **guardar los datos**. No se busca g dentro de un $\mathcal{H}$ fijo minimizando un error (no hay ERM); la hipótesis queda definida *implícitamente* por los datos más el k . Todo el trabajo se difiere a la consulta: calcular distancias, hallar los k vecinos y votar. El costo, entonces, está en la predicción, no en el entrenamiento.

8. Cotas de concentración: Markov, Chebyshev y Hoeffding

Solución. (a) Cota de Markov. Si $X \geq 0$ y $a > 0$:

$$\mathbb{E}[X] = \int_0^\infty x dP(x) \geq \int_a^\infty x dP(x) \geq a \int_a^\infty dP(x) = a \mathbb{P}[X \geq a],$$

de donde $\boxed{\mathbb{P}[X \geq a] \leq \mathbb{E}[X]/a}$. *Ejemplo:* si el ingreso promedio de una población es \$30,000, a lo sumo el 10 % gana $\geq \$300,000$, porque $\mathbb{P}[X \geq 300000] \leq 30000/300000 = 0.1$. (“Si el promedio es chico, es raro ver un gigante”).

(b) Cota de Chebyshev. Aplicamos Markov a la variable no negativa $(X - \mu)^2$ con umbral a^2 :

$$\mathbb{P}[|X - \mu| \geq a] = \mathbb{P}[(X - \mu)^2 \geq a^2] \leq \frac{\mathbb{E}[(X - \mu)^2]}{a^2} = \boxed{\frac{\sigma^2}{a^2}}.$$

(c) Cota de Hoeffding (suponiendo el Lema de Hoeffding). Sean $X_1, \dots, X_N$ i.i.d. en $[0, 1]$, $\nu = \frac{1}{N} \sum_i X_i$ y $\mu = \mathbb{E}[\nu]$. Por Chernoff, para $s > 0$,

$$\mathbb{P}[\nu - \mu \geq \epsilon] = \mathbb{P}\left[e^{s \sum_i (X_i - \mu)} \geq e^{sN\epsilon}\right] \leq e^{-sN\epsilon} \prod_i \mathbb{E}\left[e^{s(X_i - \mu)}\right].$$

El *Lema de Hoeffding* para una variable en $[a, b]$ dice $\mathbb{E}[e^{s(X - \mathbb{E}X)}] \leq e^{s^2(b-a)^2/8}$; aquí $b - a = 1$, así que cada factor es $\leq e^{s^2/8}$ y

$$\mathbb{P}[\nu - \mu \geq \epsilon] \leq e^{-sN\epsilon} e^{Ns^2/8}.$$

Minimizando el exponente $-sN\epsilon + Ns^2/8$ en s da $s = 4\epsilon$, y sustituyendo,

$$\mathbb{P}[\nu - \mu \geq \epsilon] \leq e^{-2N\epsilon^2}.$$

Por simetría (el otro lado) se duplica: $\boxed{\mathbb{P}[|\nu - \mu| \geq \epsilon] \leq 2e^{-2N\epsilon^2}}$.

9. “ μ causa ν , pero inferimos $\mu \approx \nu$ ”: ¿por qué?

Solución. Causalmente, el bin (con su fracción real μ) es quien *genera* la muestra, y de ahí sale ν : la flecha causal va $\mu \rightarrow \nu$. Pero la inferencia *invierte* esa flecha. Hoeffding nos da

$$\mathbb{P}[|\nu - \mu| \geq \epsilon] \leq 2e^{-2N\epsilon^2},$$

y lo importante es que esta cota **no depende de μ** . Como el evento “ ν y μ están lejos” es improbable, al *observar* ν (el efecto) podemos apostar a que $\mu \approx \nu$ (la causa), con confianza que crece con N . Es exactamente el sentido de *PAC*: no podemos *calcular* μ (es desconocida), pero ν es “probablemente, aproximadamente” igual a μ . Que la cota no dependa de μ es justo lo que nos autoriza a usar el ν medido como estimador del μ que no vemos. (En verificación: $\nu = E_{in}$ es una estimación honesta de $\mu = E_{out}$ para una hipótesis fijada de antemano.)

10. En $\delta = 2Me^{-2N\epsilon^2}$, ¿ ϵ o δ controla mejor a N ?

Solución. Despejamos N de $\delta = 2Me^{-2N\epsilon^2}$:

$$N = \frac{1}{2\epsilon^2} \ln \frac{2M}{\delta} = \underbrace{\frac{1}{2\epsilon^2}}_{\text{depende de } \epsilon} \left(\ln 2M + \ln \frac{1}{\delta} \right).$$

La dependencia es muy distinta en cada parámetro:

$$N \propto \frac{1}{\epsilon^2} \quad (\text{cuadrática inversa}), \quad N \propto \ln \frac{1}{\delta} \quad (\text{logarítmica}).$$

Por eso ϵ **es quien controla a N de forma mucho más fuerte**. Reducir la tolerancia a la mitad (más precisión) *cuadruplica* los datos necesarios; en cambio, exigir una confianza $10\times$ mayor (dividir δ entre 10) solo agrega $\ln 10 \approx 2.3$ dividido por $2\epsilon^2$: un pelín. En una frase: *la precisión (ϵ) es cara, la confianza (δ) es barata*. Si quiero que N no se dispare, el parámetro a cuidar es ϵ .

11. Número efectivo de rectas para separar 5 puntos en $\mathbb{R}^2$

Solución. La pregunta es cuántas *dicotomías* (etiquetados en 2 clases) de 5 puntos en posición general puede realizar una recta en $\mathbb{R}^2$. El número de dicotomías linealmente separables de N puntos en $\mathbb{R}^d$ (con sesgo) es

$$m_{\mathcal{H}}(N) = 2 \sum_{k=0}^d \binom{N-1}{k}.$$

Para $d = 2$ y $N = 5$:

$$m_{\mathcal{H}}(5) = 2 \left[\binom{4}{0} + \binom{4}{1} + \binom{4}{2} \right] = 2 [1 + 4 + 6] = \boxed{22}.$$

Es decir, de los $2^5 = 32$ etiquetados posibles, solo 22 son alcanzables con una recta; los $32 - 22 = 10$ restantes (patrones “entrelazados” tipo XOR) *ninguna* recta los separa. Esto concuerda con la función de crecimiento del perceptrón 2D y con que su *break point* es 4:

N	1	2	3	4	5
$m_{\mathcal{H}}(N)$	2	4	8	14	22
2^N	2	4	8	16	32

En $N = 4$ ya aparece la primera dicotomía imposible ($14 < 16$), y en $N = 5$ el número efectivo de rectas es 22.

12. ¿De dónde salen las junturas en una red shallow?

Solución. Salen de la **función de activación ReLU**. Cada unidad oculta calcula

$$h_j = \text{ReLU}(\theta_{j0} + \theta_{j1}x) = \max(0, \theta_{j0} + \theta_{j1}x),$$

que tiene un *quiebre* (juntura) exactamente donde su argumento cruza cero, o sea en

$$x = -\frac{\theta_{j0}}{\theta_{j1}}.$$

A la izquierda de ese punto la unidad vale 0 (plano); a la derecha, una recta. Cuando sumamos las unidades pesadas, $y = \phi_0 + \sum_j \phi_j h_j$, cada unidad aporta *su* juntura, y la salida queda *lineal a trozos* con un quiebre por unidad. Con D unidades ocultas (entrada 1D) hay hasta D junturas y $D + 1$ tramos lineales. En resumen: las junturas son hijas de la no diferenciabilidad de la ReLU en 0.

13. ¿Cuántos dobleces puedo tener aquí?

Solución. La figura es un caso 2D (entrada (x_1, x_2)) con **3 unidades ocultas**. Cada unidad ReLU se “enciende” a lo largo de una recta $\theta_{j0} + \theta_{j1}x_1 + \theta_{j2}x_2 = 0$: ese es un *doble* (un pliegue del plano). Con 3 unidades hay **3 dobleces** (3 rectas).

Esas 3 rectas en posición general parten el plano en

$$\# \text{regiones} = \sum_{j=0}^{D_i} \binom{D}{j} = \binom{3}{0} + \binom{3}{1} + \binom{3}{2} = 1 + 3 + 3 = \boxed{7},$$

(con dimensión de entrada $D_i = 2$ y $D = 3$ unidades), equivalente a la fórmula clásica $1 + L + \binom{L}{2}$ para $L = 3$ rectas. Así que: **3 dobleces** y hasta **7 regiones lineales**. Añadir más unidades agrega más pliegues (y, con profundidad, el número de regiones crece *mucho* más rápido, que es la gracia de las redes profundas).

14. ¿Por qué asumimos datos i.i.d. en aprendizaje supervisado?

Solución. Asumimos que los ejemplos $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ son **independientes e idénticamente distribuidos** por dos razones, una estadística y una computacional.

1. Idénticamente distribuidos $\Rightarrow$ la muestra representa a la población. Si el entrenamiento y el test salen de la *misma* distribución $P(x, y)$, el error en la muestra estima honestamente el error real. Es justo lo que necesitan la ley de los grandes números, Hoeffding y las cotas VC: sin “misma distribución” no hay ninguna razón para que lo aprendido se transfiera a datos nuevos. El riesgo empírico

$$R_{\text{emp}}(f) = \frac{1}{N} \sum_i \mathcal{L}(f(x_i), y_i)$$

es un estimador *insesgado* del riesgo real $R(f) = \mathbb{E}_{(x,y) \sim P}[\mathcal{L}(f(x), y)]$ precisamente porque los datos vienen de P .

2. Independientes $\Rightarrow$ la verosimilitud se factoriza. La independencia permite escribir $P(\mathcal{D}) = \prod_{i=1}^N P(x_i, y_i)$, y al tomar log el producto se vuelve *suma*: $\log P(\mathcal{D}) = \sum_i \log P(x_i, y_i)$. Eso es lo que hace tratable la estimación por máxima verosimilitud (una suma de términos por dato, en vez de una densidad conjunta de N dimensiones toda acoplada). Además, la independencia hace que la varianza del promedio empírico caiga como $1/N$, que es lo que da la concentración exponencial de Hoeffding.

¿Y si no se cumple? Con dependencia (por ejemplo, series de tiempo) o con distribución que cambia entre train y test (*dataset shift*), las garantías anteriores dejan de valer: puedes ajustar perfecto la muestra y aun así fallar afuera. Por eso el i.i.d. es el supuesto silencioso sobre el que se sostiene casi toda la teoría.

15. ¿Por qué no existe un aprendiz universal? (No Free Lunch)

Solución. La respuesta corta: porque *fuera* de los datos que viste, y sin supuestos, cualquier cosa es posible.

El teorema (No Free Lunch, Wolpert). Si promedias el error *fuera de la muestra* sobre *todas* las funciones objetivo posibles $f : \mathcal{X} \rightarrow \{0, 1\}$, todos los algoritmos de aprendizaje empatan: su desempeño promedio es el del azar,

$$\frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \mathbb{E}[L(f, \mathcal{A}(S))] = \frac{1}{2},$$

sin importar qué tan “listo” sea $\mathcal{A}$. La razón es una simetría: por cada f en la que tu algoritmo acierta en los puntos no vistos, existe su “gemela malvada” (la que invierte esas etiquetas) donde falla exactamente igual. Los datos D no determinan nada sobre lo que está fuera de D ; ese fue, de hecho, el punto de la analogía del bin.

Qué significa. No hay un algoritmo que sea el mejor para *todo* problema. La superioridad de un modelo no es una propiedad intrínseca suya, sino de qué tan bien su **sesgo inductivo** encaja con la estructura real del problema. Aprender no es “no tener supuestos”: es elegir los supuestos correctos.

La conexión con la dimensión VC. Aquí cierra con el resto del curso. Si dejaras que $\mathcal{H}$ fuera *todas* las funciones (capacidad infinita, $d_{vc} = \infty$), la cota $E_{out} \leq E_{in} + \Omega(N, d_{vc}, \delta)$ se vuelve vacía porque $\Omega \rightarrow \infty$. Por eso el aprendizaje solo es posible restringiendo $\mathcal{H}$ (VC finita) o metiendo regularidad (suavidad, simplicidad, regularización). En una frase: no hay piedra filosofal; la generalización se paga con supuestos.